Basia Kasia Asia - Dokumentacja

Forum internetowe, udostępniające klasyczne funkcje dyskusyjne: fora, wątki, komentarze, profile użytkowników oraz panel administracyjny do moderacji treści i użytkowników.

Spis treści

1. Wprowadzenie
2. Architektura systemu
3. Stos technologiczny
4. Model domenowy
5. Backend
6. API
7. Frontend
8. Bezpieczeństwo
9. Infrastruktura Azure
10. Rozwój lokalny
11. Przepływy użytkownika

1. Wprowadzenie

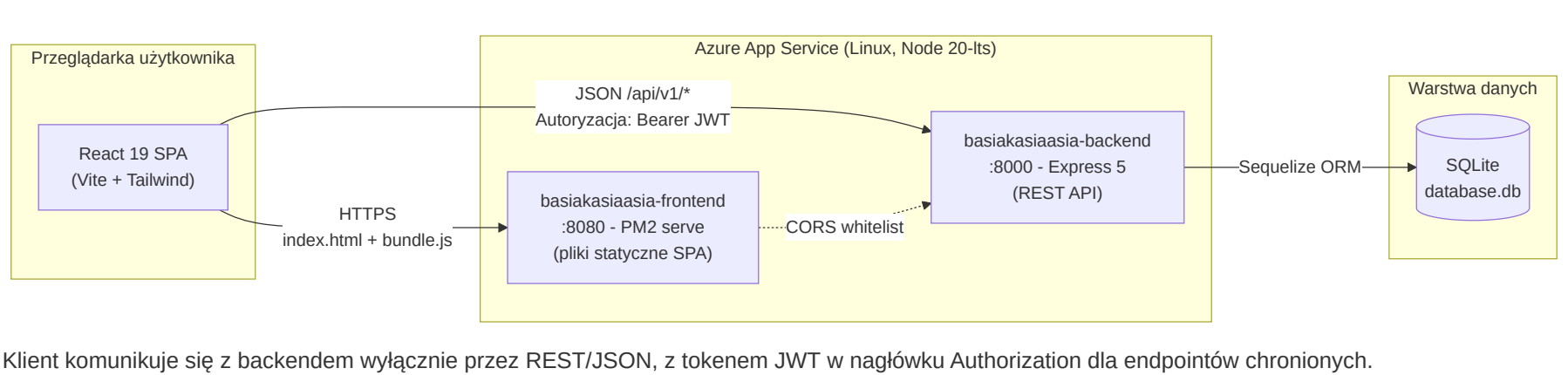
Basia Kasia Asia to aplikacja webowa typu forum, zbudowana w modelu klient-serwer i hostowana w chmurze Microsoft Azure.

Główne funkcje

- **Przeglądanie forów i wątków** bez logowania (publiczny odczyt).
- **Rejestracja i logowanie** z wykorzystaniem JWT.
- **Tworzenie wątków i komentarzy** przez zalogowanych użytkowników.
- **Komentarze zagnieżdżone** (odpowiedzi na komentarze).
- **Profile użytkowników** z bio, informacjami o użytkowniku.
- **Panel administracyjny** z zarządzaniem użytkownikami i forami.
- **Baza danych** z rolami, przykładowymi użytkownikami, forami oraz wątkami.

2. Architektura systemu

System ma architekturę **trójwarstwową** z podziałem na warstwę prezentacji (SPA), logiki biznesowej (REST API) oraz danych (SQLite).
Obie aplikacje webowe (frontend i backend) są hostowane jako osobne usługi *Azure App Service* w ramach jednego *App Service Plan*.



Przeglądarka użytkownika
React 19 SPA
(Vite + Tailwind)
Azure App Service (Linux, Node 20-lts)
JSON /api/v1/*
Autoryzacja: Bearer JWT
HTTPS
index.html + bundle.js
basiakasiaasia-frontend
:8080 - PM2 serve
(pliki statyczne SPA)
CORS whitelist
basiakasiaasia-backend
:8000 - Express 5
(REST API)
Sequelize ORM
Warstwa danych
SQLite
database.db

Klient komunikuje się z backendem wyłącznie przez REST/JSON, z tokenem JWT w nagłówku Authorization dla endpointów chronionych.
Backend jest bezstanowy - cała sesja trzymana jest po stronie klienta w sessionStorage.

3. Stos technologiczny

Backend

Warstwa	Technologia	Wersja
Runtime	Node.js	20-lts
Framework HTTP	Express	^5.1.0
Język	TypeScript	^5.9.3
ORM	Sequelize + sequelize-typescript	^6.37.7 / ^2.1.6
Baza danych	SQLite (sqlite3)	^5.1.7
Autentykacja	jsonwebtoken, bcrypt	^9.0.2 / ^6.0.0
Logi	Winston + Morgan	^3.18.3 / ^1.10.1
CORS	cors	^2.8.5
Dev	tsx (watch), ESLint, Prettier	^4.20.6

Frontend

Warstwa	Technologia	Wersja
Runtime	React + React DOM	^19.1.1
Bundler	Vite	^7.1.7
Routing	React Router / React Router DOM	^7.9.4
Język	TypeScript	~5.9.3
Stylowanie	TailwindCSS 4	^4.1.16
UI	Radix UI (15+ komponentów)	najnowsze
Rich-text	Lexical	^0.37.0
HTTP	Axios	^1.12.2
Tabele	TanStack Table	^8.21.3
Powiadomienia	Sonner	^2.0.7
Ikony	Lucide / Lucide-React	^0.548.0
Daty	date-fns, react-day-picker	^4.1.0 / ^9.11.1

Infrastruktura

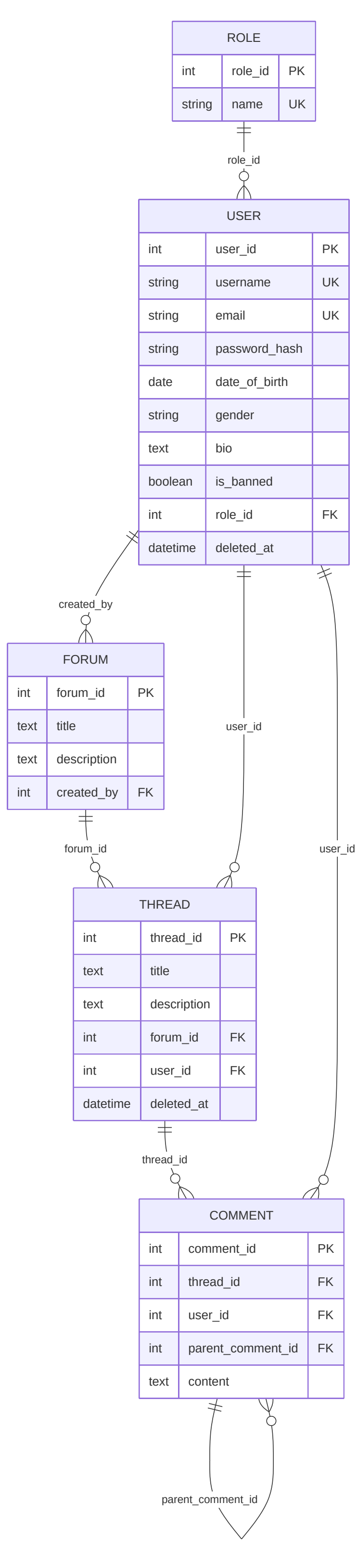
Element	Technologia
IaC	Terraform ≥ 1.7 (provider azurerm ~4.0)
Hosting	Azure App Service - Linux, Node 20-lts, SKU B1

Narzędzia dodatkowe

Element	Technologia
Środowisko developerskie	nix (dev-shell z Node.js i Prettier)
Środowisko do IaC	nix (dev-shell z Azure CLI i Terraform)

4. Model domenowy

Model bazy danych składa się z pięciu encji powiązanych relacjami 1:N oraz jednej relacji rekurencyjnej (Comment -> parent_comment_id).
ORM Sequelize odzwierciedla je jako klasy z dekoratorami (@Table, @ForeignKey, @BelongsTo, @HasMany).



ROLE
int role_id PK
string name UK
role_id
USER
int user_id PK
string username UK
string email UK
string password_hash
date date_of_birth
string gender
text bio
boolean is_banned
int role_id FK
datetime deleted_at
created_by
user_id
user_id
FORUM
int forum_id PK
text title
text description
int created_by FK
forum_id
THREAD
int thread_id PK
text title
text description
int forum_id FK
int user_id FK
datetime deleted_at
thread_id
COMMENT
int comment_id PK
int thread_id FK
int user_id FK
int parent_comment_id FK
text content
parent_comment_id

Znaczenie relacji

- **Role -> User (1:N)** - każdy użytkownik ma dokładnie jedną rolę; domyślnie user.
- **User -> Forum (1:N)** - autor (created_by) forum; widoczny w panelu admina.
- **Forum -> Thread (1:N)** - wątek należy do jednego forum; forum_id jest indeksowane.
- **User -> Thread (1:N)** - autor wątku; soft-delete przez deleted_at.
- **Thread -> Comment (1:N)** - komentarze pierwszego poziomu w wątku.
- **Comment -> Comment (1:N, self-ref)** - odpowiedzi na komentarz; buduje drzewo dyskusji.

Seed data

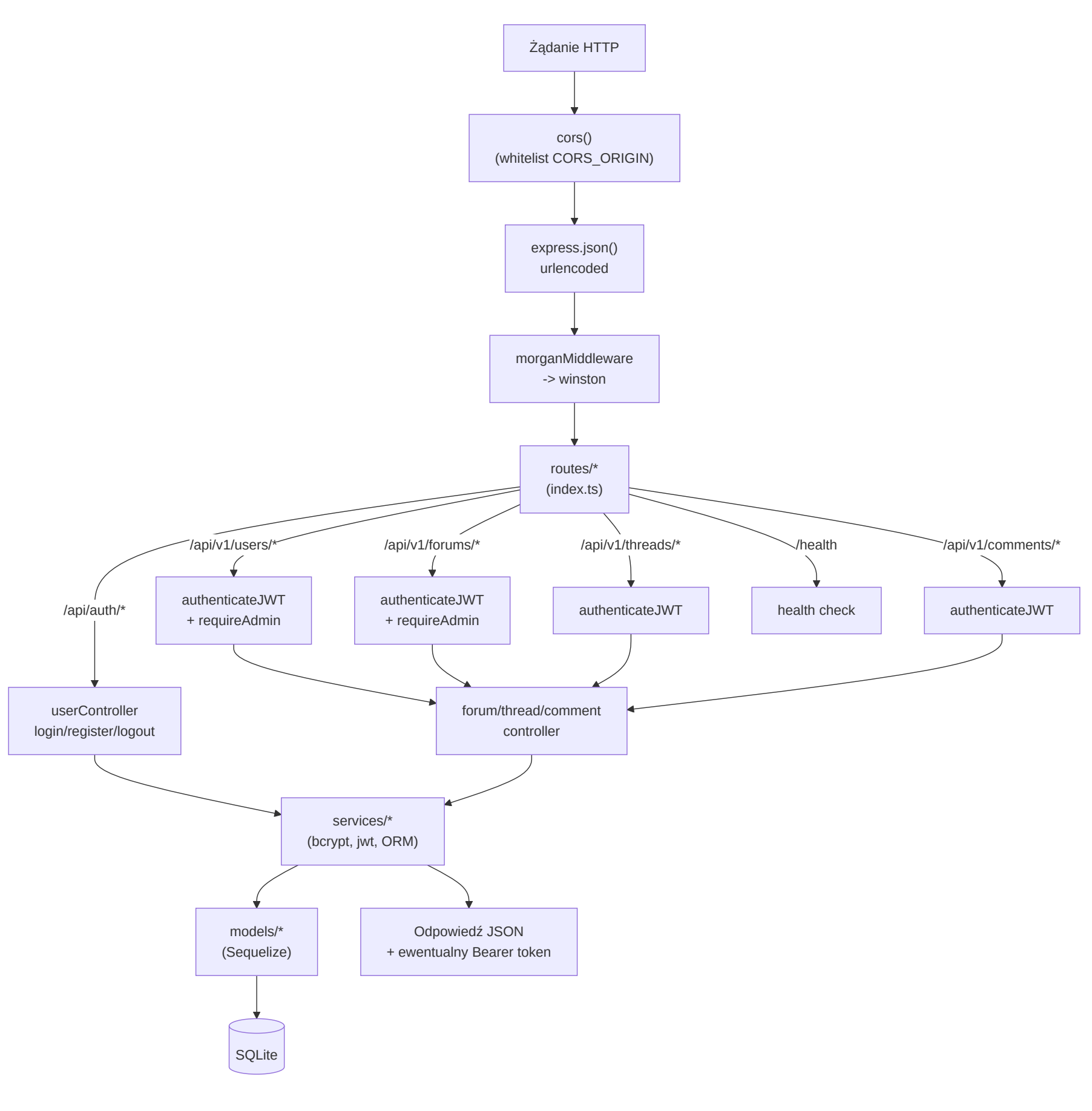
backend/src/seed/seedData.ts wypełnia bazę przykładowymi rekordami tylko przy pierwszym uruchomieniu:

- 3 role: user, moderator, admin.
- 10 użytkowników.
- 3 fora.
- 10 wątków.
- Komentarze z odpowiedziami do każdego wątku.

Domyślne konto administratora jest tworzone w na podstawie zmiennych środowiskowych.

5. Backend

Warstwy i przepływ żądania



Żądanie HTTP
cors()
(whitelist CORS_ORIGIN)
express.json()
urlencoded
morganMiddleware
-> winston
routes/*
(index.ts)
/api/auth/*
/api/v1/users/*
/api/v1/forums/*
/api/v1/threads/*
/health
/api/v1/comments/*
authenticateJWT
+ requireAdmin
authenticateJWT
+ requireAdmin
authenticateJWT
health check
authenticateJWT
userController
login/register/logout
forum/thread/comment
controller
services/*
(bcrypt, jwt, ORM)
models/*
(Sequelize)
Odpowiedź JSON
+ ewentualny Bearer token
SQLite

Bootstrap (src/index.ts)

startServer() wykonuje cztery kroki:

1. connectDatabase() - uwierzytelnienie Sequelize + sync() + ewentualny seed.
2. ensureDefaultRoles() - tworzy role bazowe, jeśli brak.
3. ensureDefaultAdmin() - tworzy konto admin z DEFAULT_ADMIN_PASSWORD.
4. app.listen(PORT) - domyślnie port 8000.

CORS akceptuje listę originów z CORS_ORIGIN.

Logi trafiają do logs/ (Winston: error.log, combined.log) i na konsolę.
Zapytania SQL logowane są tylko w środowisku developerskim przez Morgan.

6. API

Wszystkie endpointy (poza /health i /api/auth/*) są udostępniane pod /api/v1.

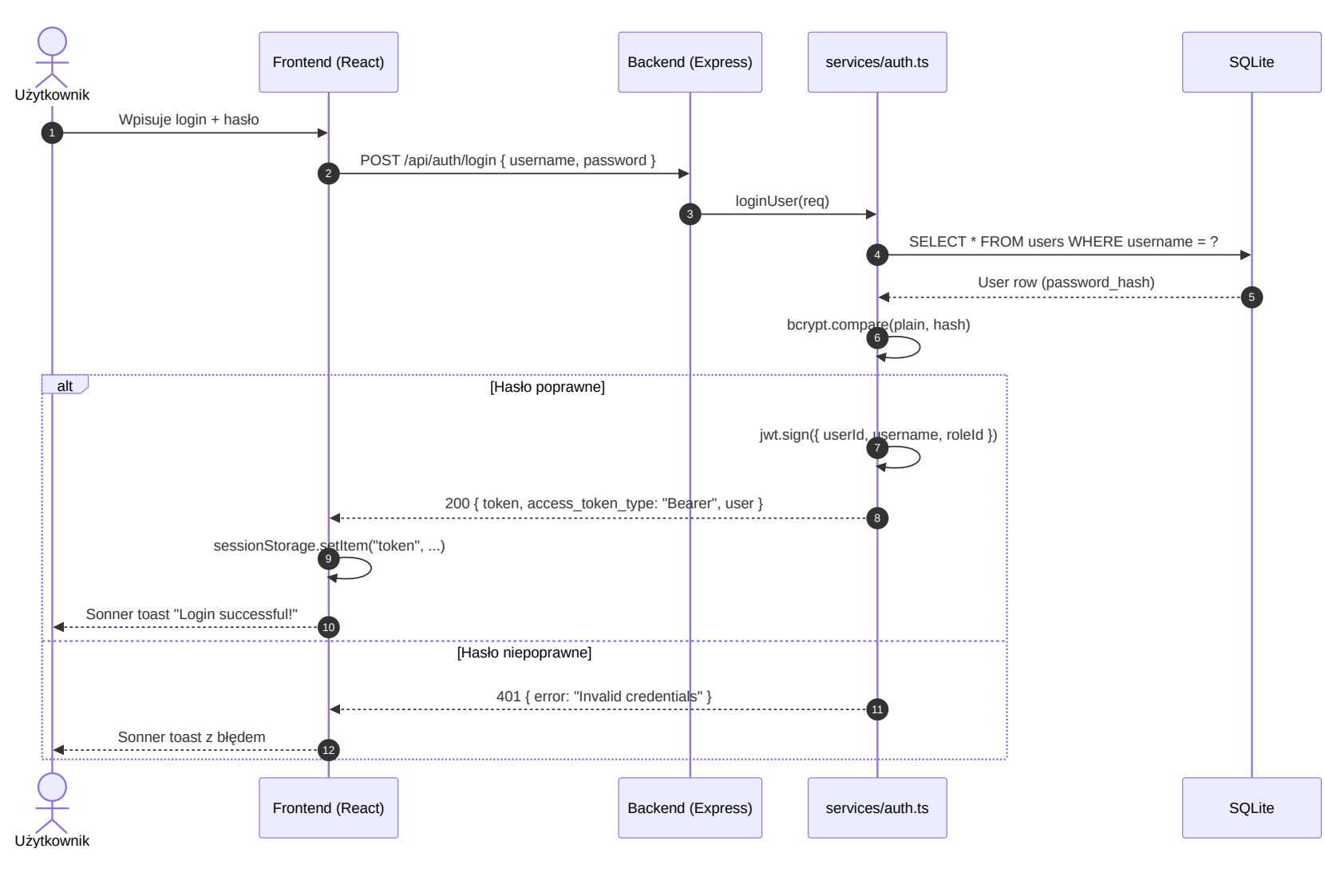
Tabela endpointów

Metoda	Ścieżka	Auth	Opis
GET	/health	-	Health check + wersja API
POST	/api/auth/register	-	Rejestracja
POST	/api/auth/login	-	Logowanie, zwraca JWT
POST	/api/auth/logout	-	Unieważnienie sesji
GET	/api/v1/me	JWT	Profil zalogowanego użytkownika
GET	/api/v1/users	JWT + admin	Lista wszystkich użytkowników
GET	/api/v1/users/id/:id	JWT + admin	Użytkownik po ID
GET	/api/v1/users/nickname/:nickname	JWT + admin	Użytkownik po nicku
PUT	/api/v1/users/:id	JWT	Aktualizacja profilu
DELETE	/api/v1/users/:id	JWT	Soft-delete użytkownika
POST	/api/v1/users/:id/ban	JWT + admin	Zbanowanie
POST	/api/v1/users/:id/unban	JWT + admin	Odbanowanie
GET	/api/v1/forums	-	Lista forów
GET	/api/v1/forums/:id	-	Forum po ID
GET	/api/v1/categories/:categoryId/forums	-	Fora w kategorii
POST	/api/v1/forums	JWT + admin	Utworzenie forum
PUT	/api/v1/forums/:id	JWT + admin	Aktualizacja forum
DELETE	/api/v1/forums/:id	JWT + admin	Usunięcie forum
GET	/api/v1/threads	-	Lista wątków
GET	/api/v1/threads/:id	-	Wątek po ID
GET	/api/v1/forums/:forumId/threads	-	Wątki w forum
GET	/api/v1/users/:userId/threads	-	Wątki użytkownika
POST	/api/v1/threads	JWT	Utworzenie wątku
PUT	/api/v1/threads/:id	JWT	Aktualizacja wątku
DELETE	/api/v1/threads/:id	JWT	Soft-delete wątku
GET	/api/v1/threads/:threadId/comments	-	Komentarze w wątku
GET	/api/v1/threads/:threadId/comments/stats	-	Statystyki komentarzy
GET	/api/v1/comments/:id	-	Komentarz po ID
GET	/api/v1/comments/:commentId/replies	-	Odpowiedzi na komentarz
GET	/api/v1/users/:userId/comments	-	Komentarze użytkownika
POST	/api/v1/comments	JWT	Utworzenie komentarza (z opcjonalnym parent_comment_id)
PUT	/api/v1/comments/:id	JWT	Aktualizacja komentarza
DELETE	/api/v1/comments/:id	JWT	Usunięcie komentarza

Middleware autoryzacji

- authenticateJWT - weryfikuje Authorization: Bearer przez (jwt.verify + secret). W razie błędu zwraca 401.
- requireRole(roleId) / requireAdmin (requireRole(3)) - sprawdza req.user.roleId; brak roli => 403 Forbidden.
- optionalAuthentication - parsuje token, jeśli obecny, ale nie wymaga go.

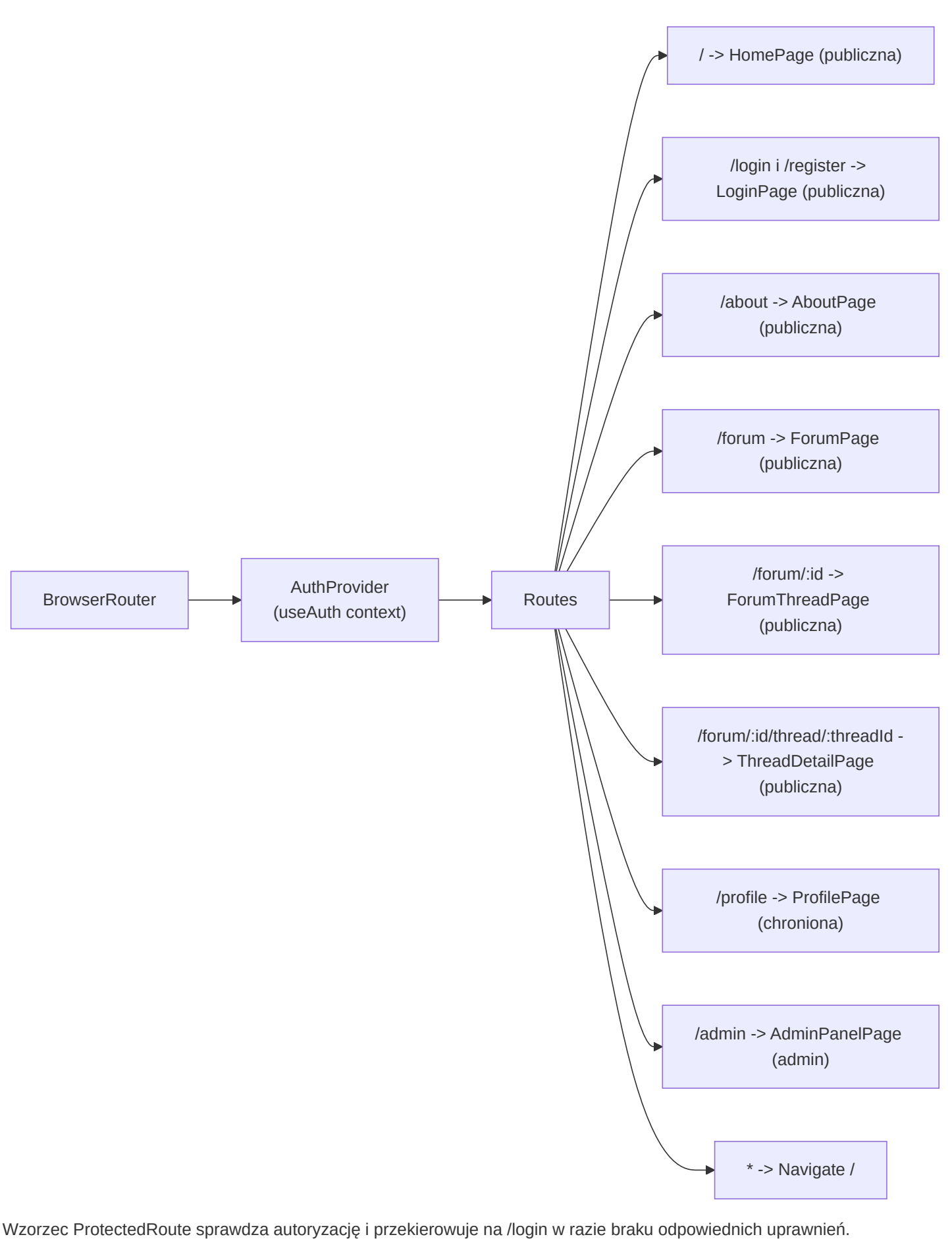
Przepływ logowania



Użytkownik
Frontend (React)
Backend (Express)
services/auth.ts
SQLite
1
Wpisuje login + hasło
2
POST /api/auth/login { username, password }
3
loginUser(req)
4
SELECT * FROM users WHERE username = ?
5
User row (password_hash)
6
bcrypt.compare(plain, hash)
alt
[Hasło poprawne]
7
jwt.sign({ userId, username, roleId })
8
200 { token, access_token_type: "Bearer", user }
9
sessionStorage.setItem("token", ...)
10
Sonner toast "Login successful!"
[Hasło niepoprawne]
11
401 { error: "Invalid credentials" }
12
Sonner toast z błędem
Użytkownik
Frontend (React)
Backend (Express)
services/auth.ts
SQLite

7. Frontend

Routing i ochrona tras



BrowserRouter
AuthProvider
(useAuth context)
Routes
/ -> HomePage (publiczna)
/login i /register -> LoginPage (publiczna)
/about -> AboutPage (publiczna)
/forum -> ForumPage (publiczna)
/forum/:id -> ForumThreadPage (publiczna)
/forum/:id/thread/:threadId -> ThreadDetailPage (publiczna)
/profile -> ProfilePage (chroniona)
/admin -> AdminPanelPage (admin)
* -> Navigate /

Wzorzec ProtectedRoute sprawdza autoryzację i przekierowuje na /login w razie braku odpowiednich uprawnień.

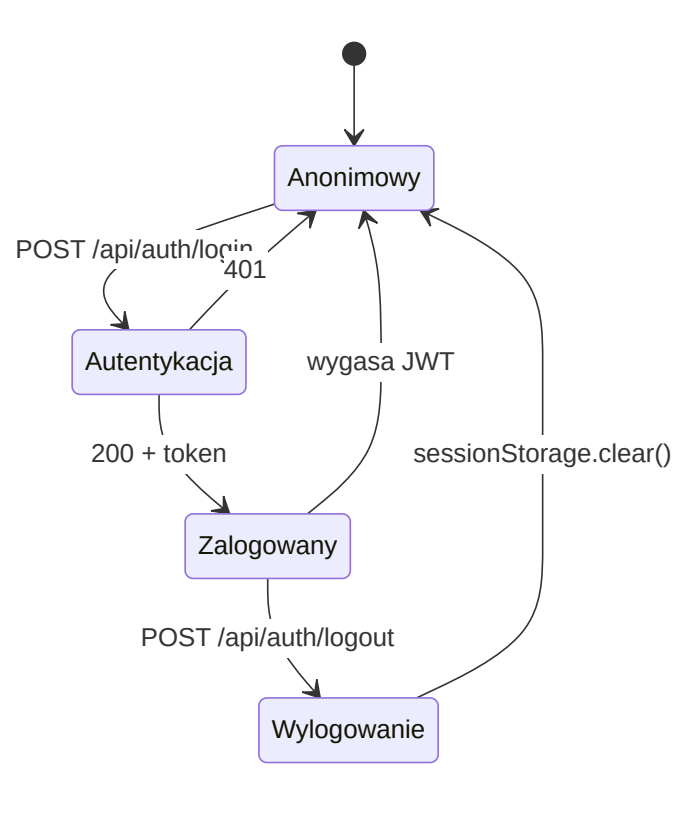
Klient HTTP i interceptor

services/api.ts tworzy instancję axios z baseURL = ${VITE_API_BASE_URL}/api/v1.
Interceptor request automatycznie dodaje nagłówek:

```
config.headers['Authorization'] = ${access_token_type} ${token};
```

gdzie token i access_token_type są przechowywane w sessionStorage.

State machine autoryzacji

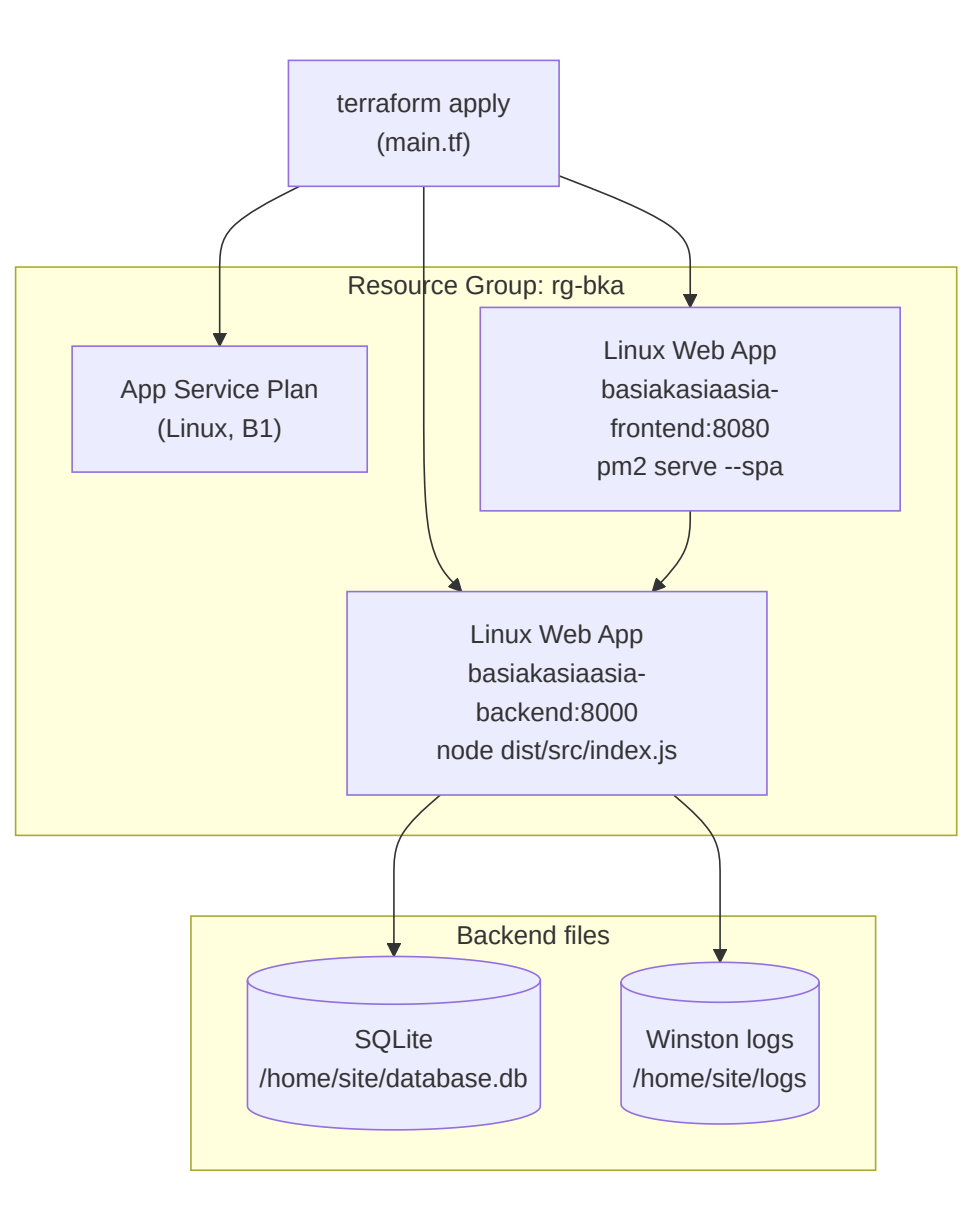


Anonimowy
POST /api/auth/login
401
Autentykacja
wygasa JWT
200 + token
sessionStorage.clear()
Zalogowany
POST /api/auth/logout
Wylogowanie

8. Bezpieczeństwo

- **Hasła:** bcrypt z kosztem 10.
- **JWT:** jsonwebtoken, algorytm domyślny (HS256), JWT_SECRET pobierany z env, czas życia domyślnie 24h.
  Claims: { userId, username, email, roleId }.
- **RBAC:** warstwa middleware/auth.ts pilnuje endpointów administracyjnych.
- **CORS:** whitelist originów z CORS_ORIGIN. Na Azure ustawiony na hostname frontendu, więc przeglądarka nie zezwoli na obce domeny.
- **Walidacja danych wejściowych:** podstawowa w kontrolerach (parsowanie req.body, sprawdzanie Object.keys, walidacja długości).
- **Bezpieczeństwo transportu:** HTTPS wymuszany przez Azure App Service (domyślny certyfikat *.azurewebsites.net).
- **Soft-delete:** wątki i użytkownicy nie są kasowani fizycznie - ustawiane jest deleted_at.

9. Infrastruktura Azure



terraform apply
(main.tf)
Resource Group: rg-bka
App Service Plan
(Linux, B1)
Linux Web App
basiakasiaasia-
frontend:8080
pm2 serve --spa
Linux Web App
basiakasiaasia-
backend:8000
node dist/src/index.js
Backend files
SQLite
/home/site/database.db
Winston logs
/home/site/logs

Deploy

Proces wdrożenia składa się z kroków:

1. **terraform init && terraform apply --auto-approve** - tworzy RG, plan, oba Web Appy.
2. Skrypt **update-backend** - buduje backend (npm ci && npm run build), pakuje wymagane pliki w zip i publikuje przez az webapp deploy.
3. Skrypt **update-frontend** - buduje frontend (npm ci && npm run build) analogicznie pakuje wymagane pliki i publikuje aplikację.

Alternatywnie **update-app** wywołuje oba update-*.sh po kolei.

10. Rozwój lokalny

Wymagania

- Node.js 20.x.
- npm 10+ (lockfile package-lock.json).
- Azure CLI + Terraform.

Pliki flake.nix zawierają definicję środowiska developerskiego.
.envrc automatycznie ładuje zmienne środowiskowe przy odpowiednio skonfigurowanym direnv.

Pierwsza konfiguracja

Wypełnienie plików .env na podstawie .env.example w obu katalogach:

```
# 1. Backend
cp backend/.env.example backend/.env
# 2. Frontend
cp frontend/.env.example frontend/.env
```

(Opcjonalnie) Wejście w nix-shell

```
nix develop
```

Zmienne środowiskowe

backend/.env:

- PORT=8000
- NODE_ENV=development
- JWT_SECRET=change-this-in-production
- JWT_EXPIRES_IN=24h
- DEFAULT_ADMIN_PASSWORD=admin123
- DB_PATH=backend/db/database.db
- CORS_ORIGIN=http://localhost:5173

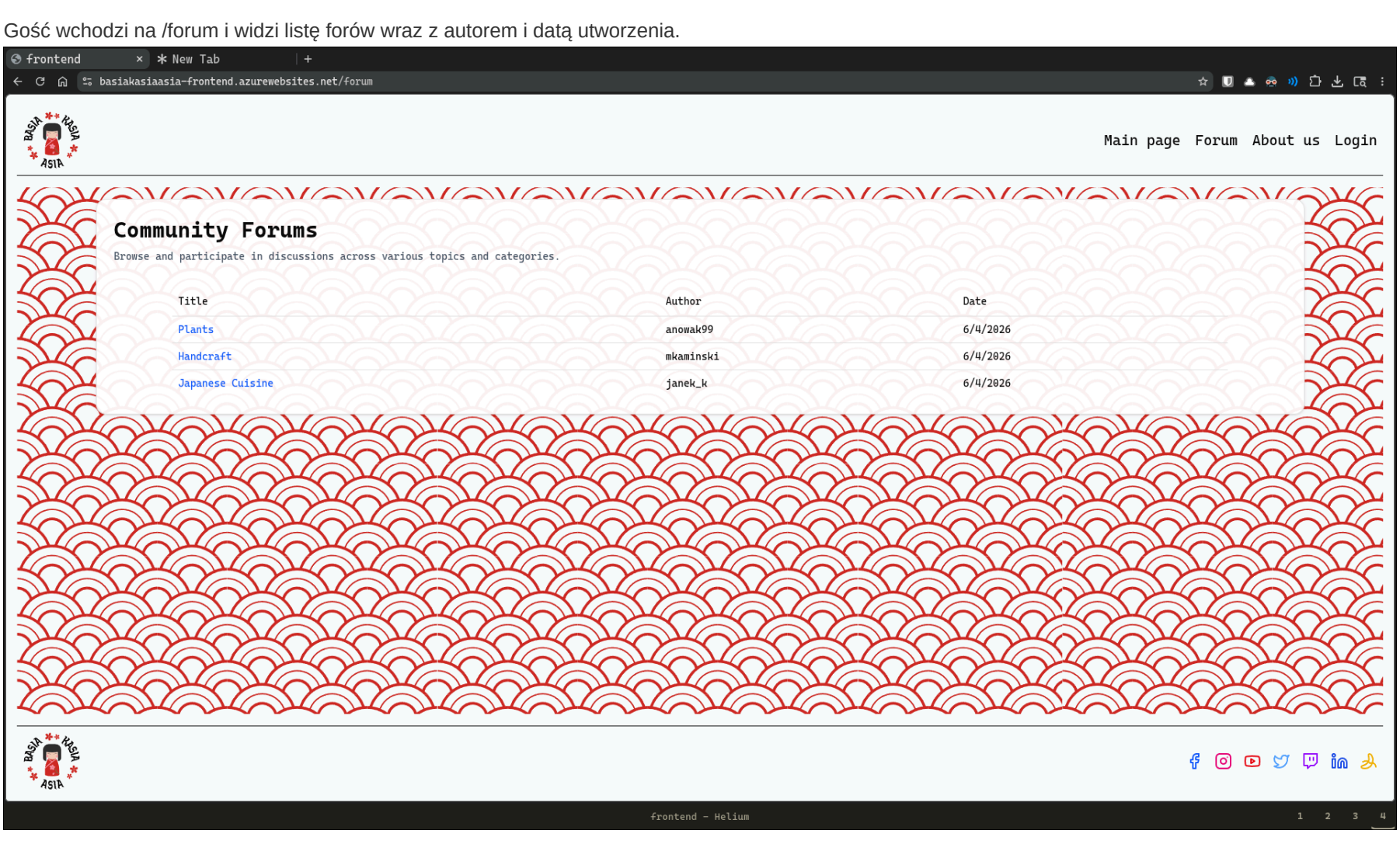
frontend/.env:

- VITE_API_BASE_URL=http://localhost:8000

11. Przepływy użytkownika

11.1. Przeglądanie forów

Gość wchodzi na /forum i widzi listę forów wraz z autorem i datą utworzenia.



frontend
New Tab
basiakasiaasia-frontend.azurewebsites.net/forum
BASIA KASIA ASIA
Main page
Forum
About us
Login
frontend — Helium

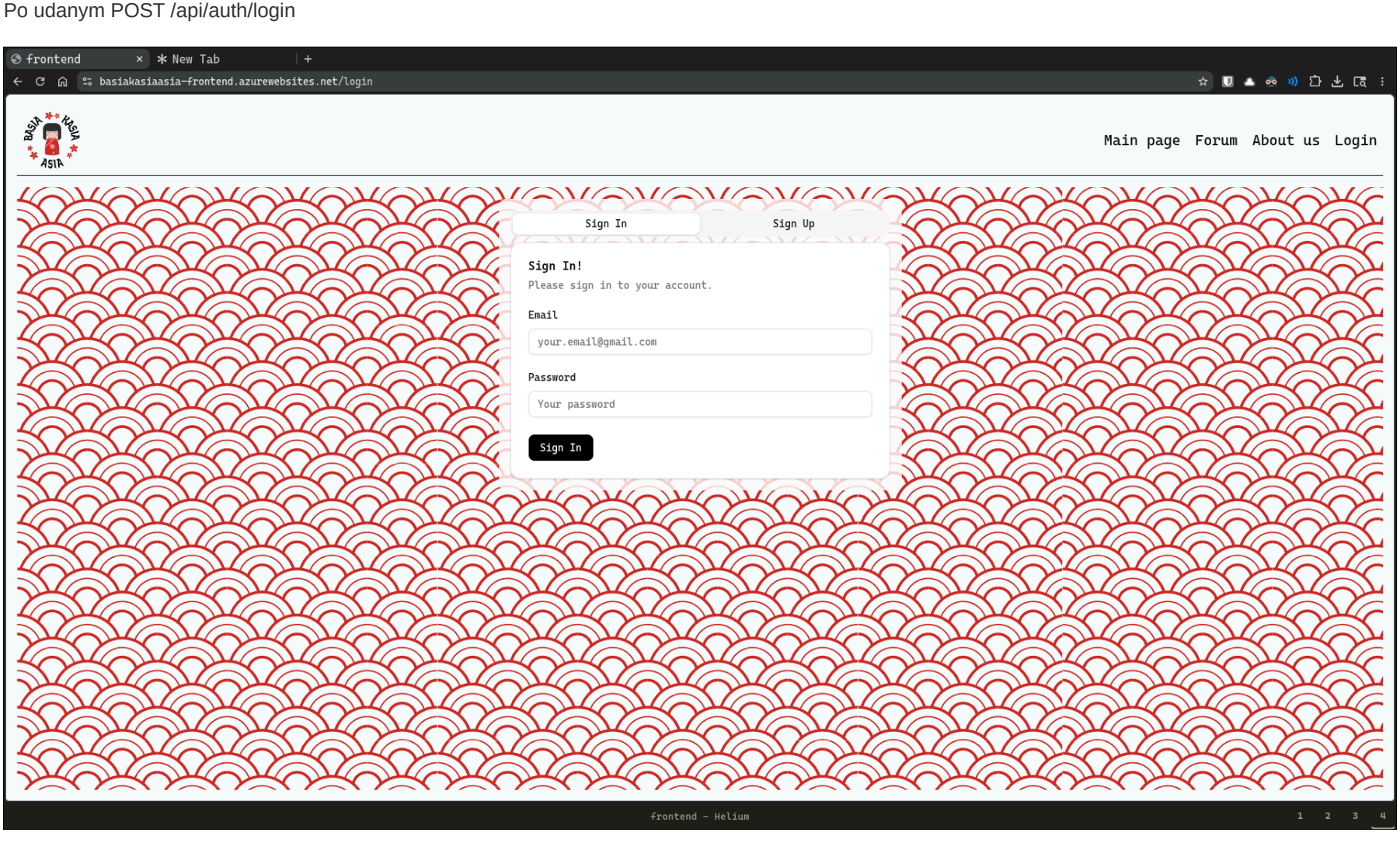
Community Forums

Browse and participate in discussions across various topics and categories.

Title	Author	Date
Plants	anowak99	6/4/2026
Handcraft	mkaminski	6/4/2026
Japanese Cuisine	janek_k	6/4/2026

11.2. Logowanie i toast

Po udanym POST /api/auth/login



frontend
New Tab
basiakasiaasia-frontend.azurewebsites.net/login
BASIA KASIA ASIA
Main page
Forum
About us
Login
Sign In
Sign Up
Sign In
frontend — Helium

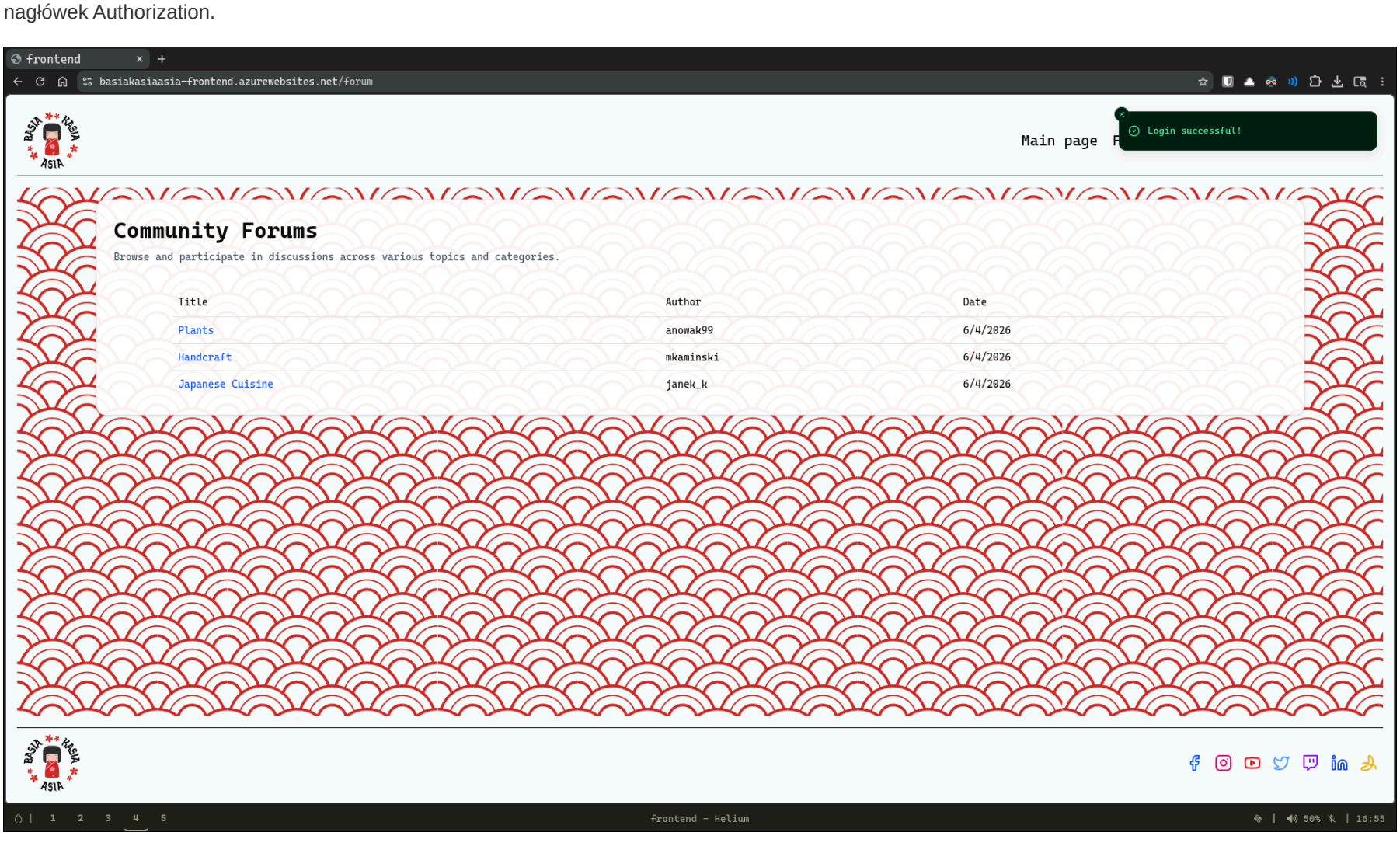
Sign In!

Please sign in to your account.

Email: your.email@gmail.com

Password: Your password

frontend wyświetla toast Sonner *"Login successful!"* i zapisuje token w sessionStorage. Od tego momentu interceptory axios automatycznie dołączają nagłówek Authorization.



frontend
basiakasiaasia-frontend.azurewebsites.net/forum
BASIA KASIA ASIA
Main page
frontend — Helium

Login successful!

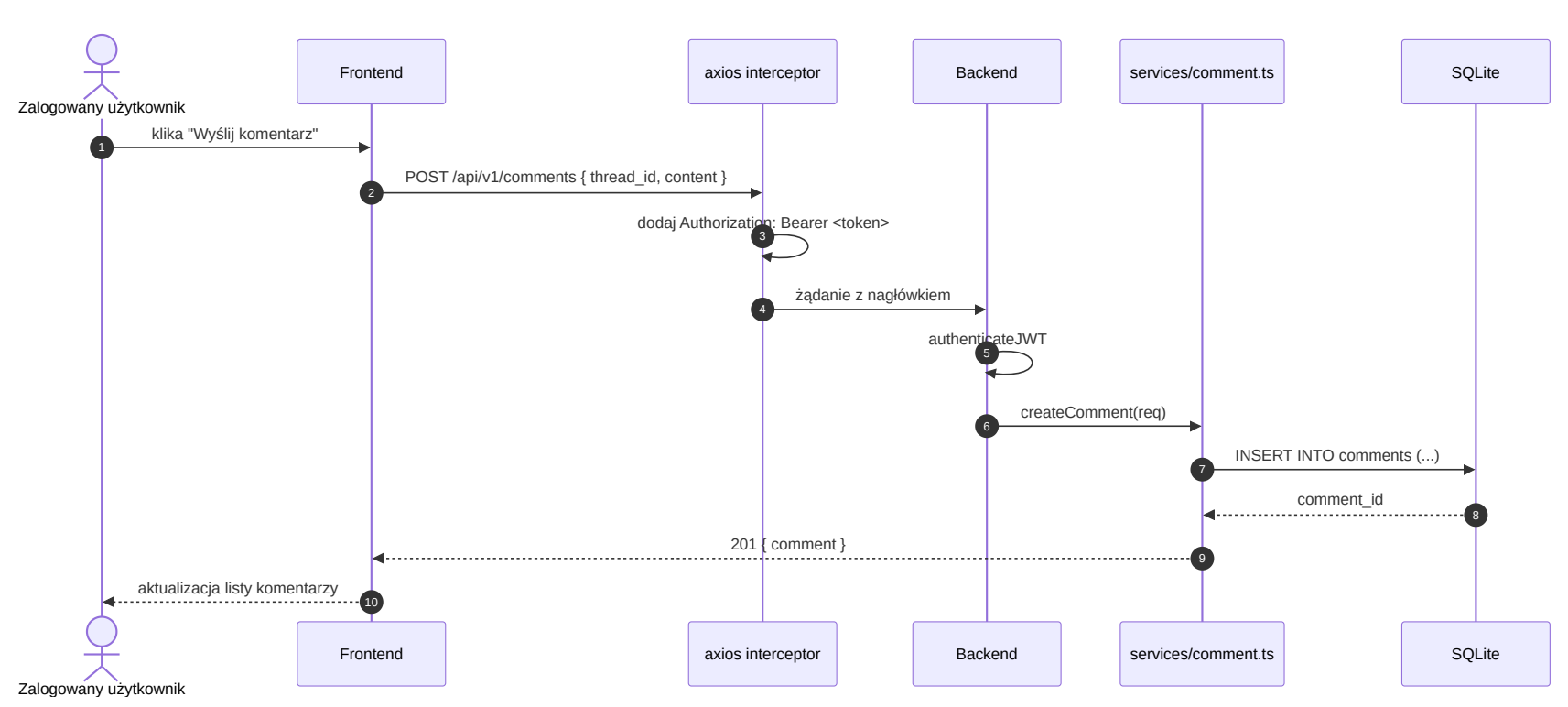
Community Forums

Browse and participate in discussions across various topics and categories.

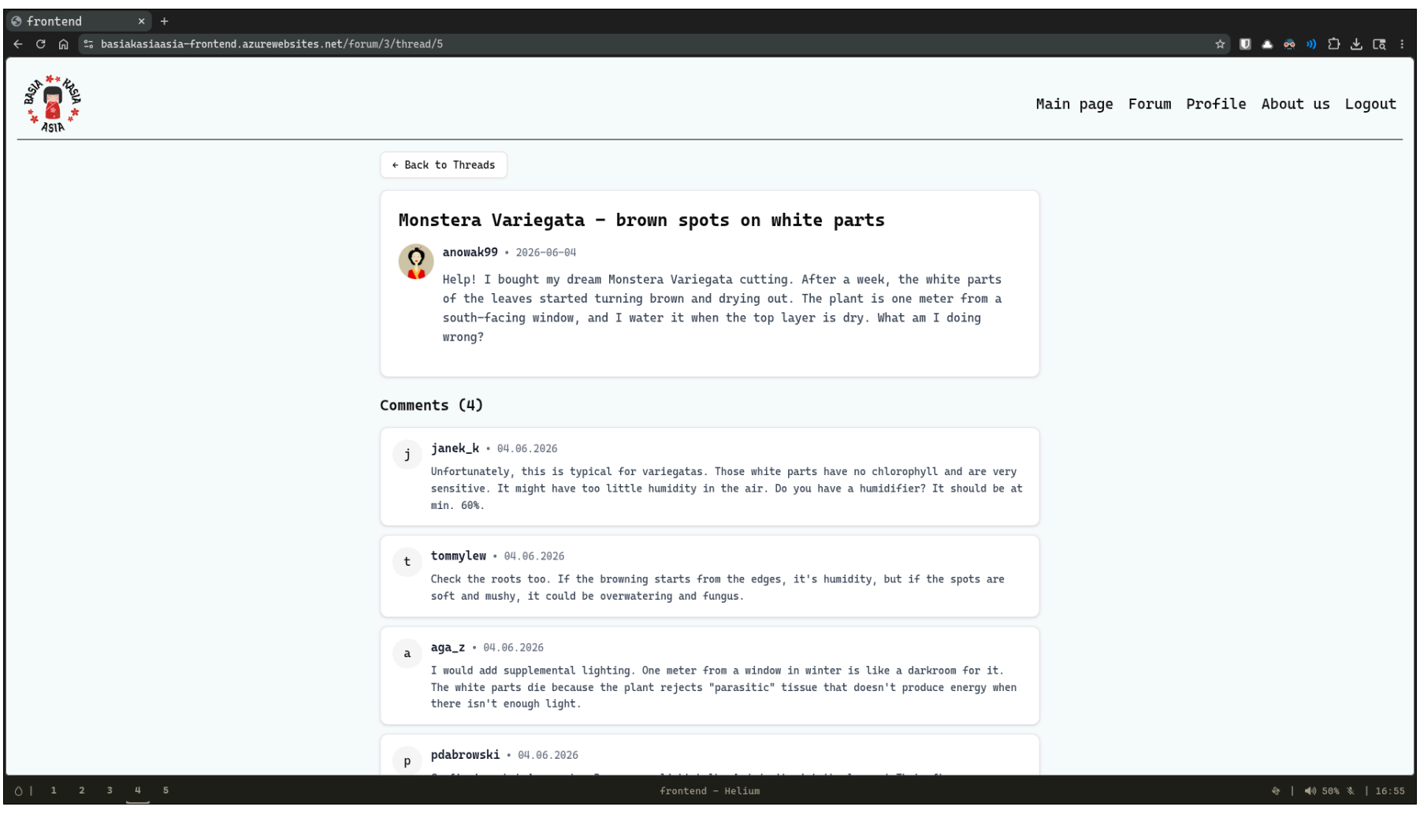
Title	Author	Date
Plants	anowak99	6/4/2026
Handcraft	mkaminski	6/4/2026
Japanese Cuisine	janek_k	6/4/2026

11.3. Czytanie wątku i komentarze

Wątek pobiera dane z GET /api/v1/threads/:id, a komentarze z GET /api/v1/threads/:threadId/comments. Renderowany jest autor, data, treść pierwszego posta oraz drzewo komentarzy z awatarami.



Zalogowany użytkownik
Frontend
axios interceptor
Backend
services/comment.ts
SQLite
1
klika "Wyślij komentarz"
2
POST /api/v1/comments { thread_id, content }
3
dodaj Authorization: Bearer <token>
4
żądanie z nagłówkiem
5
authenticateJWT
6
createComment(req)
7
INSERT INTO comments (...)
8
comment_id
9
201 { comment }
10
aktualizacja listy komentarzy
Zalogowany użytkownik
Frontend
axios interceptor
Backend
services/comment.ts
SQLite



frontend
basiakasiaasia-frontend.azurewebsites.net/forum/3/thread/5
BASIA KASIA ASIA
Main page
Forum
Profile
About us
Logout
← Back to Threads
frontend — Helium

Monstera Variegata – brown spots on white parts

anowak99 • 2026-06-04

Help! I bought my dream Monstera Variegata cutting. After a week, the white parts of the leaves started turning brown and drying out. The plant is one meter from a south-facing window, and I water it when the top layer is dry. What am I doing wrong?

Comments (4)

janek_k • 04.06.2026

Unfortunately, this is typical for variegatas. Those white parts have no chlorophyll and are very sensitive. It might have too little humidity in the air. Do you have a humidifier? It should be at min. 60%.

tommylew • 04.06.2026

Check the roots too. If the browning starts from the edges, it's humidity, but if the spots are soft and mushy, it could be overwatering and fungus.

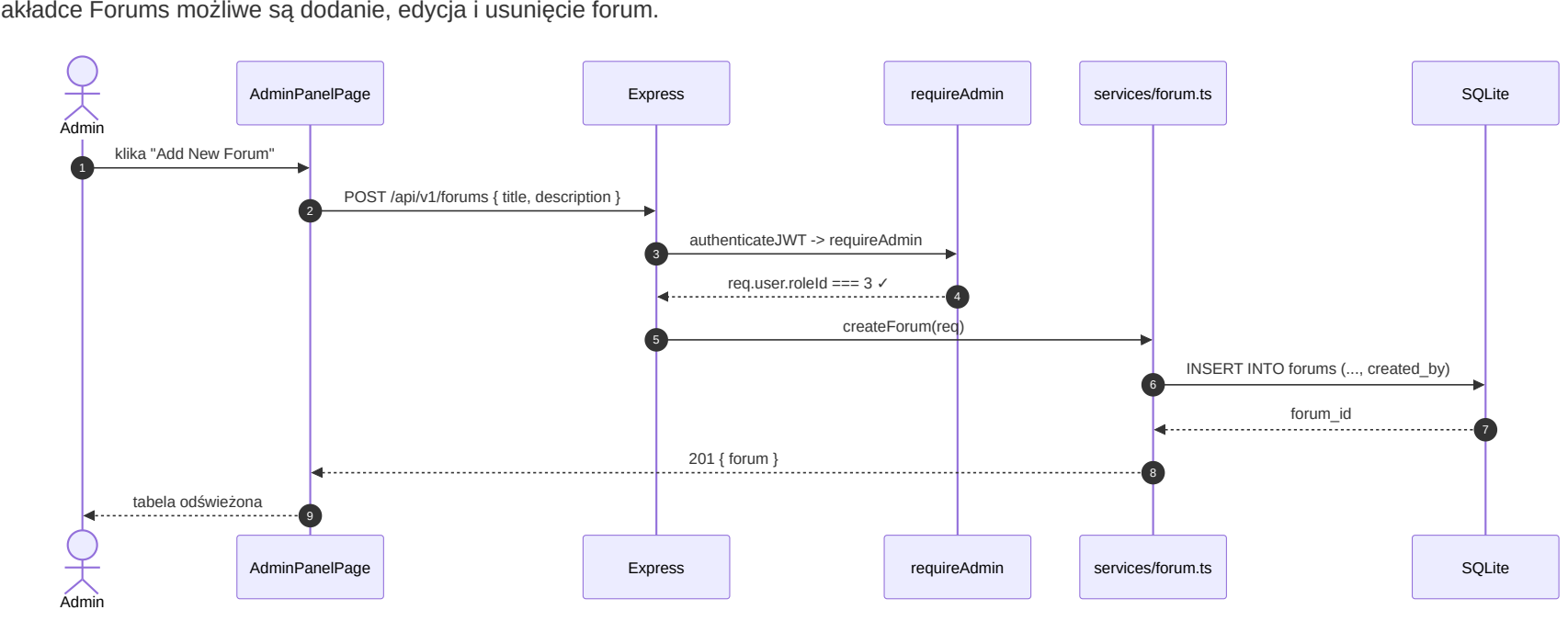
aga_z • 04.06.2026

I would add supplemental lighting. One meter from a window in winter is like a darkroom for it. The white parts die because the plant rejects "parasitic" tissue that doesn't produce energy when there isn't enough light.

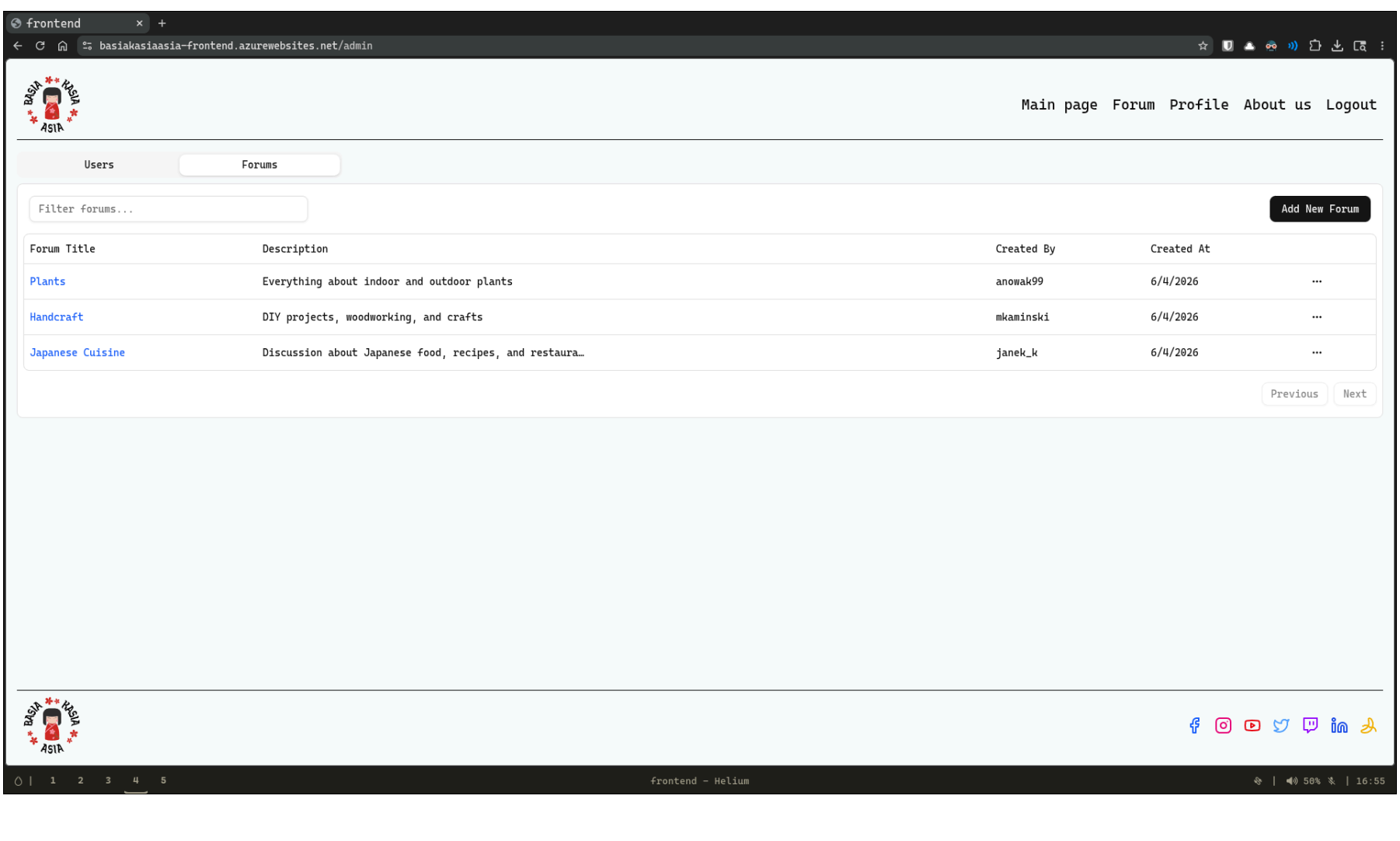
pdabrowski • 04.06.2026

11.4. Panel administracyjny

Administrator wchodzi na /admin. Strona posiada dwie zakładki **Users** i **Forums**.
W zakładce Users możliwe jest banowanie, odbanowywanie i dodanie nowych administratorów.
W zakładce Forums możliwe są dodanie, edycja i usunięcie forum.



Admin
AdminPanelPage
Express
requireAdmin
services/forum.ts
SQLite
1
klika "Add New Forum"
2
POST /api/v1/forums { title, description }
3
authenticateJWT -> requireAdmin
4
req.user.roleId === 3 ✓
5
createForum(req)
6
INSERT INTO forums (..., created_by)
7
forum_id
8
201 { forum }
9
tabela odświeżona
Admin
AdminPanelPage
Express
requireAdmin
services/forum.ts
SQLite



frontend
basiakasiaasia-frontend.azurewebsites.net/admin
BASIA KASIA ASIA
Main page
Forum
Profile
About us
Logout
Users
Forums
Filter forums...
Add New Forum
Previous
Next
frontend — Helium

Forum Title	Description	Created By	Created At
Plants	Everything about indoor and outdoor plants	anowak99	6/4/2026
Handcraft	DIY projects, woodworking, and crafts	mkaminski	6/4/2026
Japanese Cuisine	Discussion about Japanese food, recipes, and restaura…	janek_k	6/4/2026